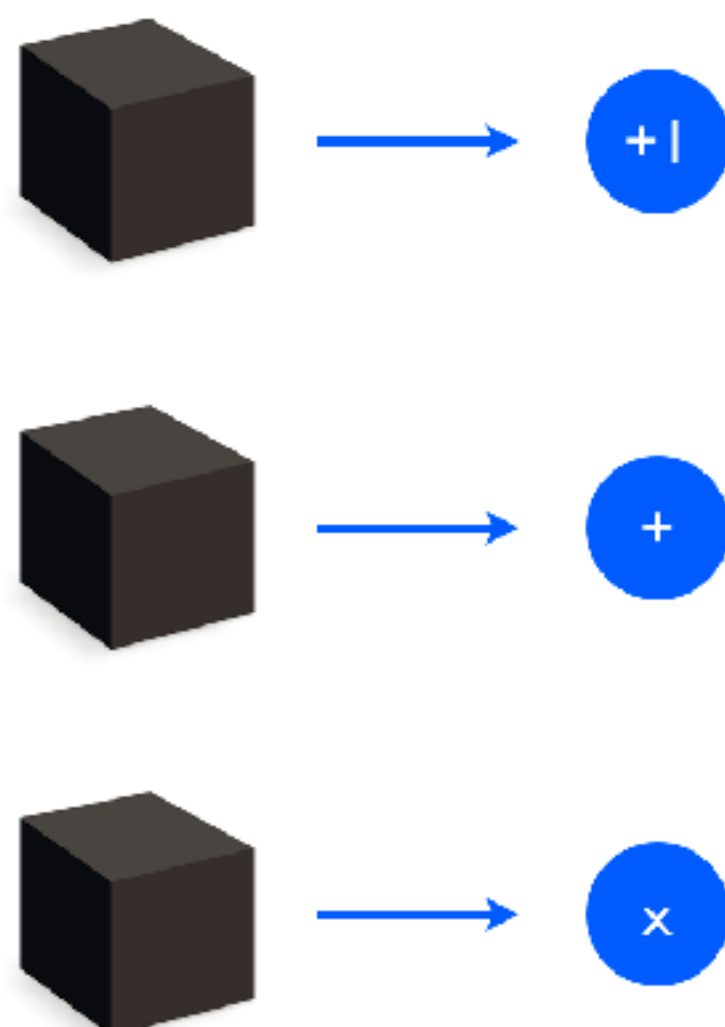
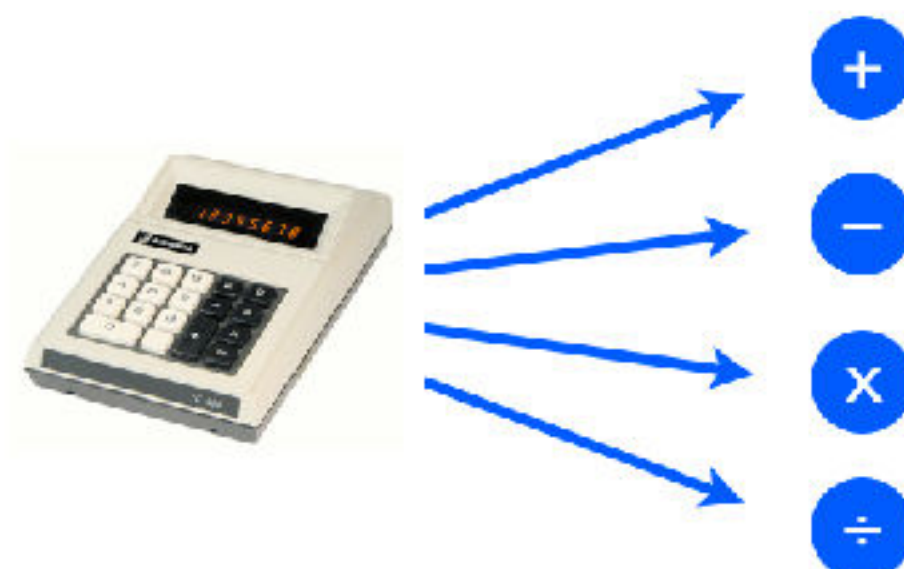


37. Machines and Programs

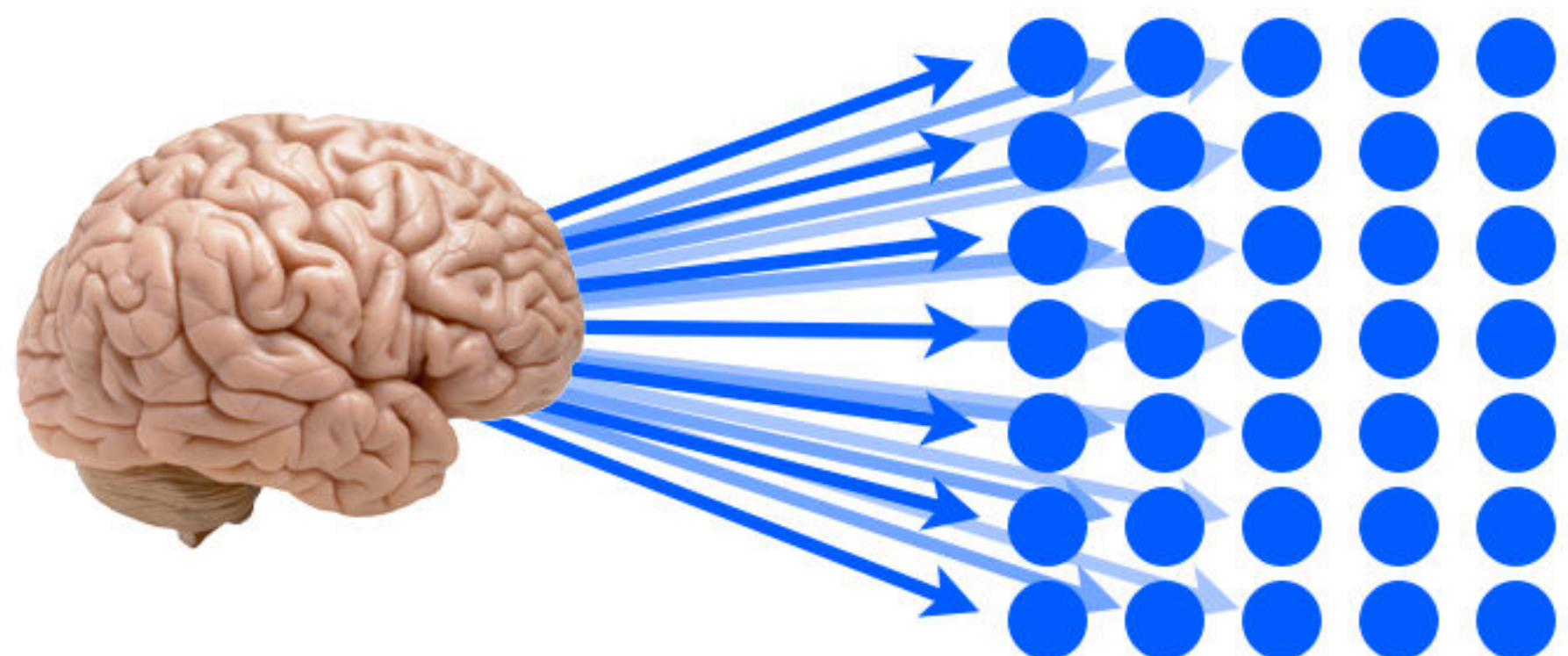
Multi-function Machines



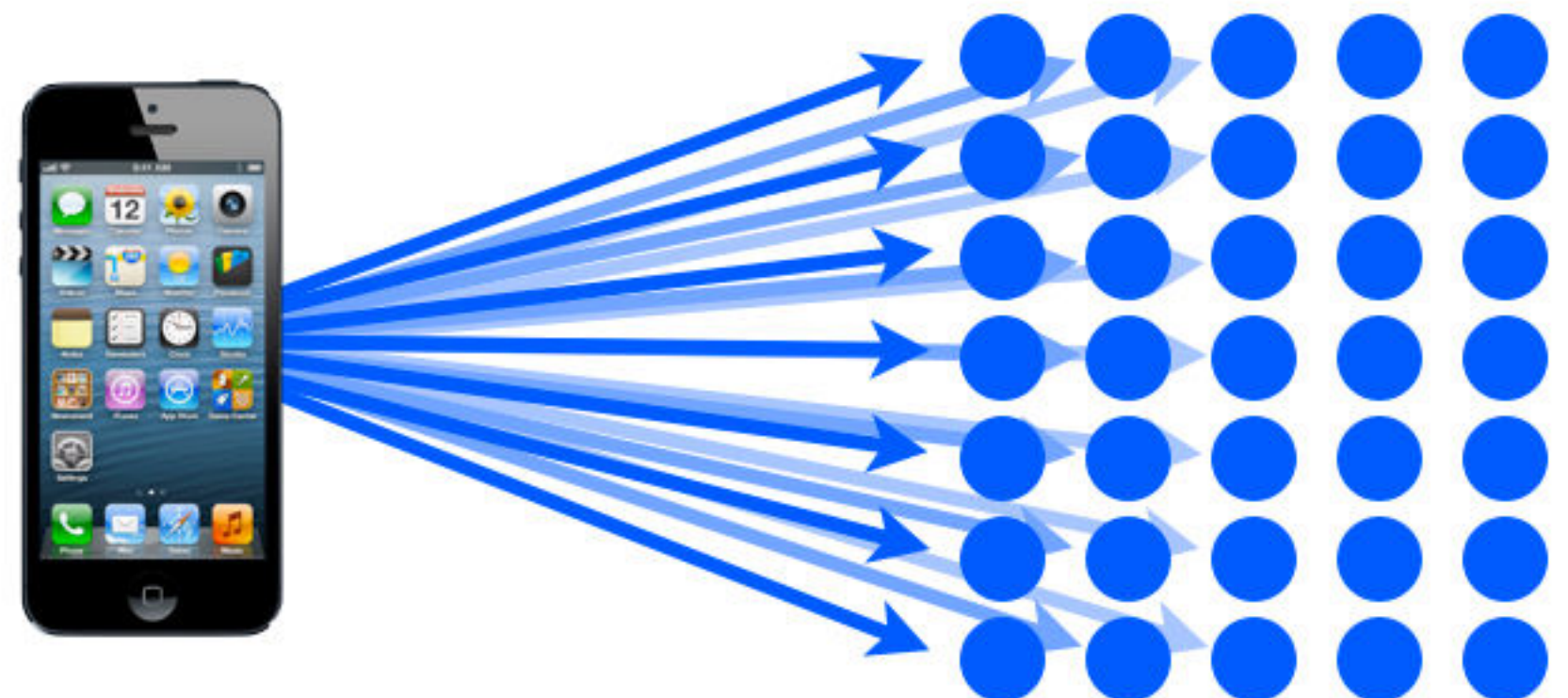
So far, all of the machines we have encountered compute one function each. Of course, it is possible to construct a machine which computes more than one function. A simple calculator computes four functions.



Our brains can compute a lot more than four functions. In fact, it seems like there is no practical limit on the number of functions a brain could compute. It is an *infinite* function machine.

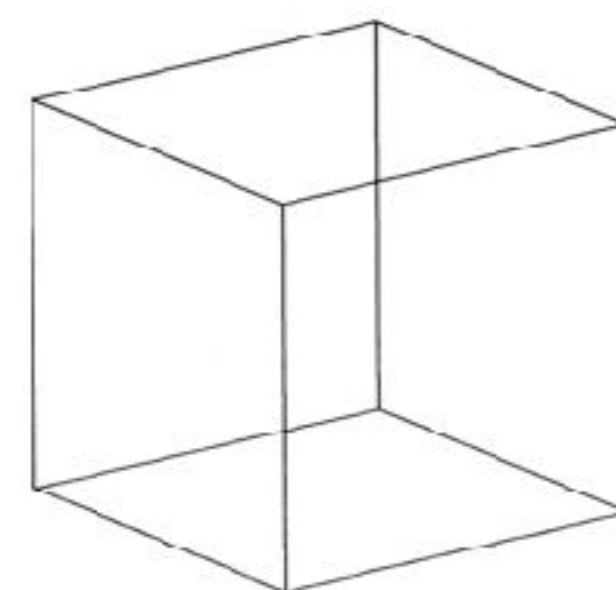


This isn't just true of our brains. It's true of any computer which can read *programs*. Such devices are known as **universal computers**. Your laptop and your phone are universal computers.



Universal Turing Machines: the goal

The goal: Create a Turing Machine which can perform the task of any other Turing Machine. This is called a **Universal Turing Machine (UTM)**.



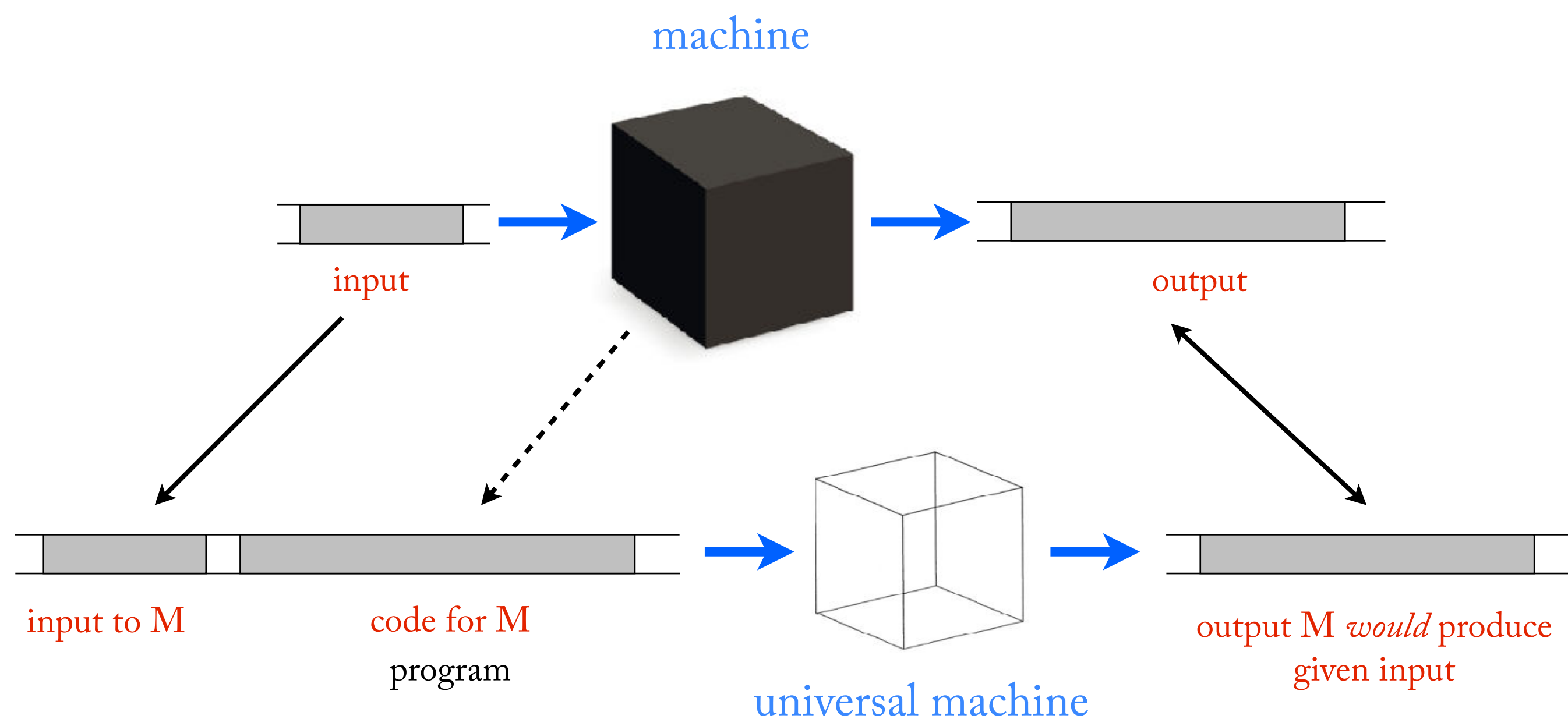
universal machine

Universal Turing Machines: the idea

Strategy for designing a UTM:

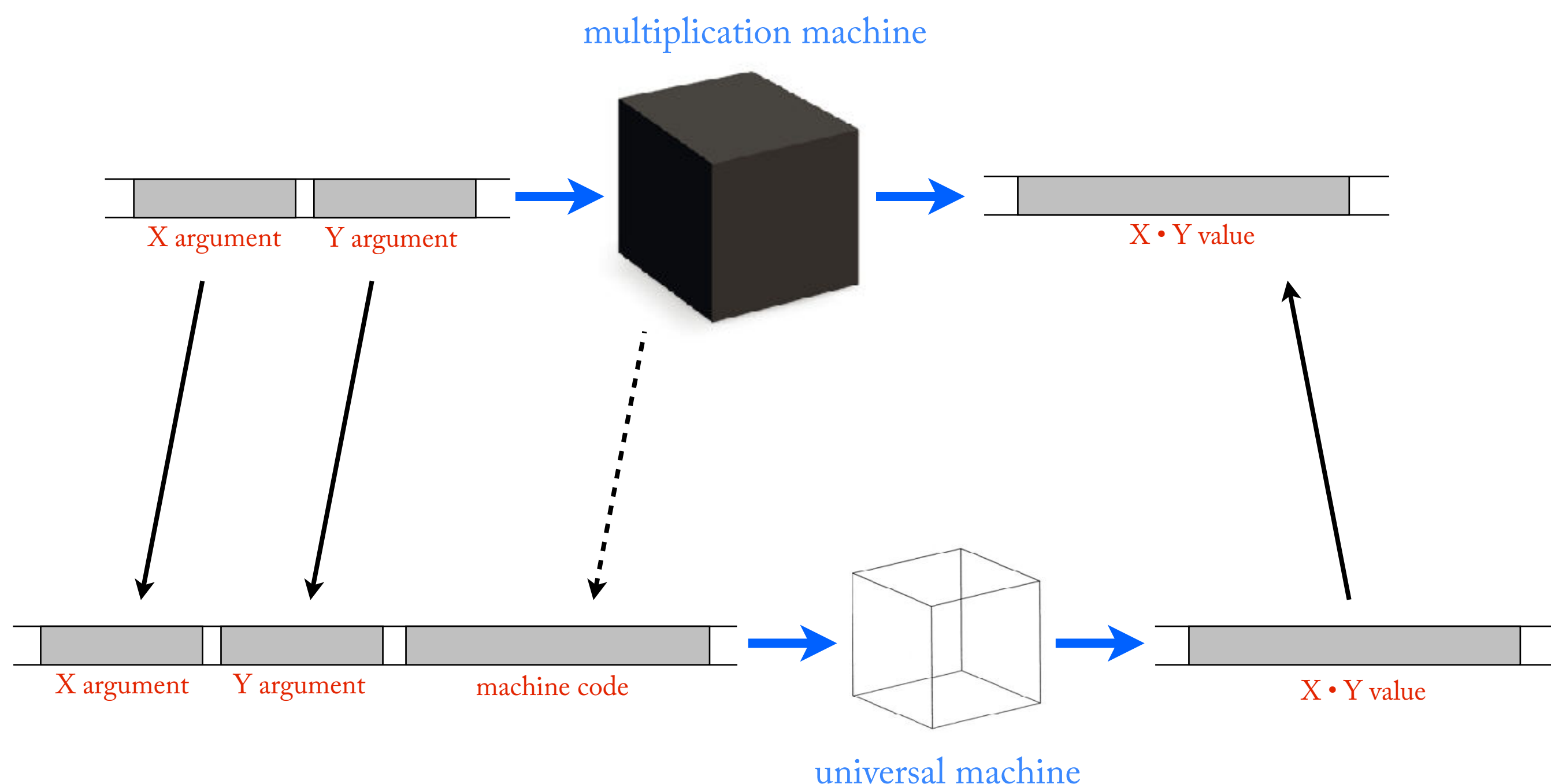
Stage 1: find a method for turning an arbitrary Turing Machine (the simulated machine) into something a Turing Machine can read— i.e. 1's and 0's on tape. This something is called the simulated machine's **code number**.

Stage 2: invent a UTM which takes as input (i) any simulated machine's code number, and (ii) the simulated machine's input; the UTM then outputs the simulated machine's output.



For example, let M be a multiplication machine. It takes two inputs (corresponding to the X and Y arguments) and returns one output (corresponding to the value $X \cdot Y$). And let's say that M 's code number is C .

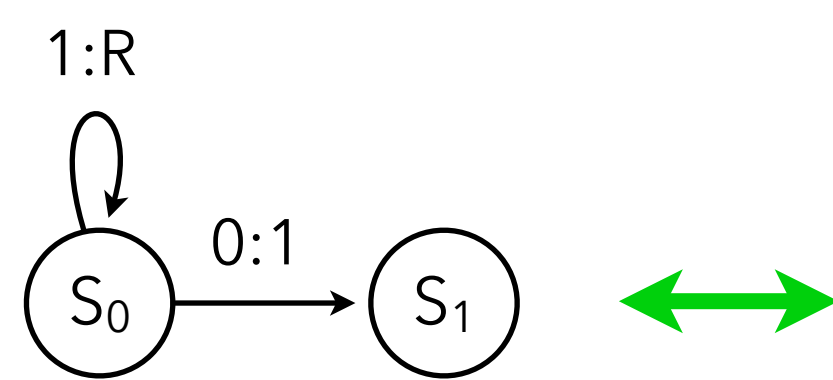
Then if we give a tape which contains, X , Y , and C to the UTM, it will return the value $X \cdot Y$.



Codes Numbers (aka Programs)

Derive a machine's code number in five easy steps.

Step 1. Choose a Turing Machine, and draw up it's state table.



STATE	IN	OUT	NEXT STATE
S ₀	1	R	S ₀
	0	1	S ₁
S ₁	1	H	
	0	H	

Step 2. Divide each state cell into rows according to input. Eliminate rows which do not lead to another state.

STATE	IN	OUT	NEXT STATE
S ₀	1	R	S ₀
S ₀	0	1	S ₁
S ₁	1	H	
S ₁	0	H	

Step 3. Replace all symbols with numbers, according to a translation system.

State S _n	→	n
No state	→	0
Read a "0"	→	0
Read a "1"	→	1
Write a "0"	→	0
Write a "1"	→	1
Move left (L)	→	2
Move right (R)	→	3
Halt (H)	→	4

STATE	IN	OUT	NEXT STATE
0	1	3	0
0	0	1	1
1	1	4	0
1	0	4	0

Step 4. Represent the table by a sequence of quadruples:

state, in, out, next
state, in, out, next
etc...

0, 1, 3, 0
0, 0, 1, 1
1, 1, 4, 0
1, 0, 4, 0

Step 5. Represent each number in tally. Represent commas with 0's and line breaks with *s. The result is a string 0's and 1's.

001011100*000010
1*1010111100*100
0111100

Now every single Turing Machine can be associated with a code number. And every single (well-formed) code number corresponds to a Turing Machine.



0001 0001 0000 0011 0000 0101 0101 0110 0111

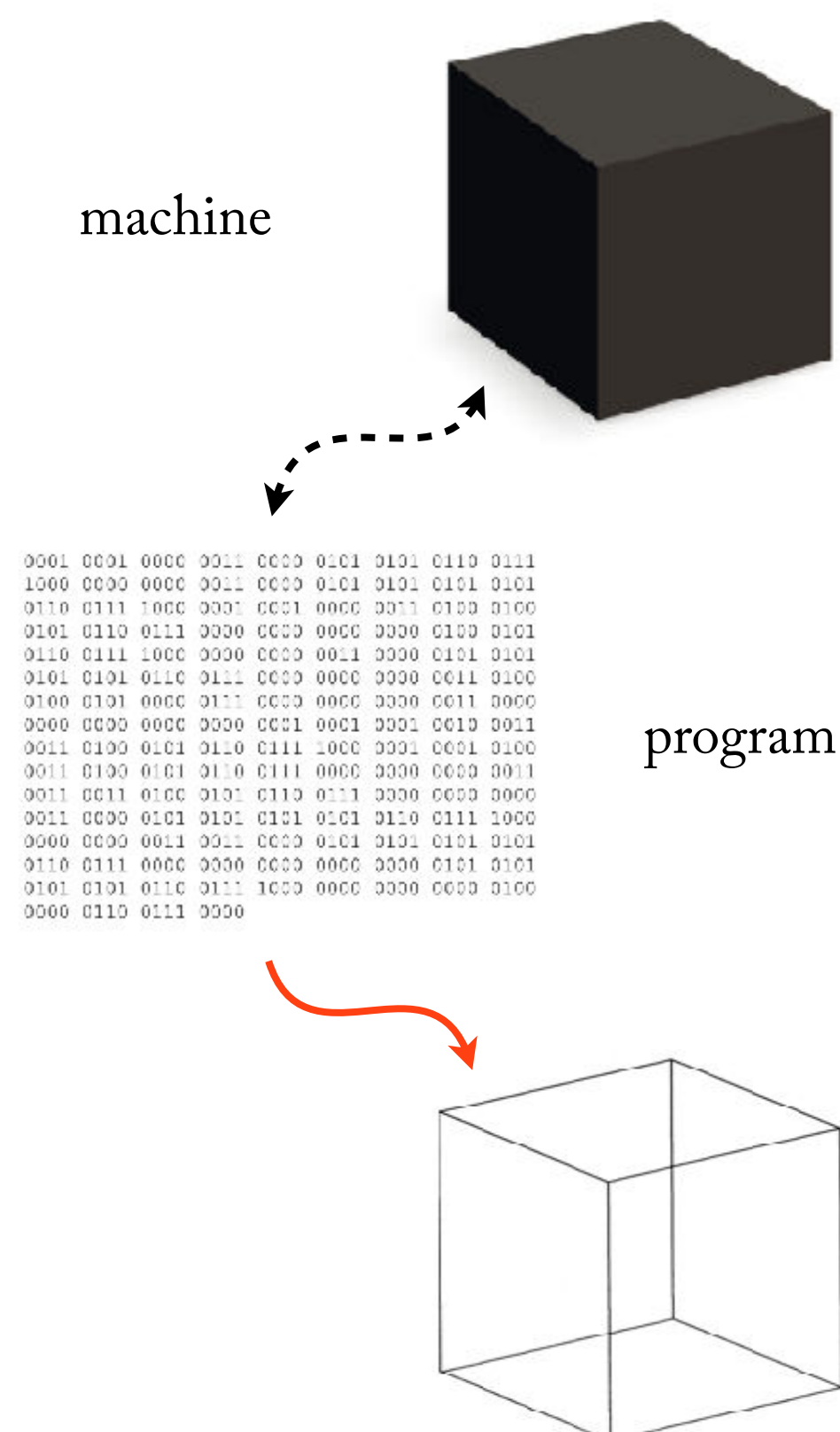
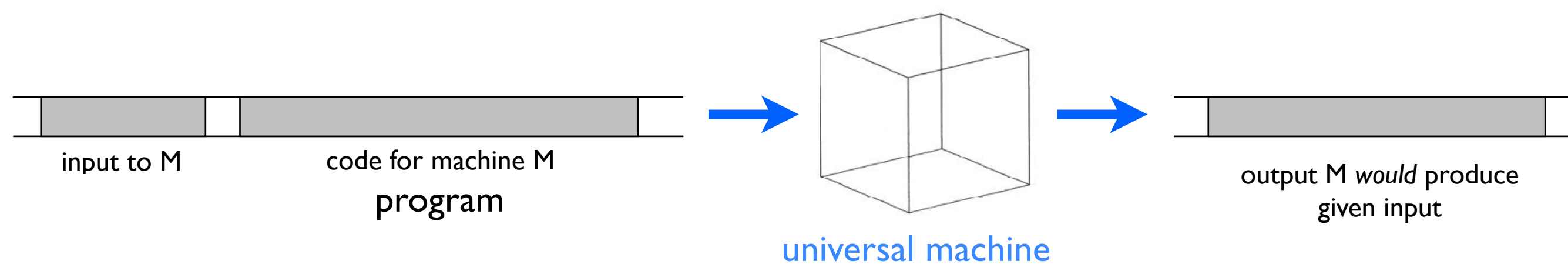


0110 0111 1000 0001 0001 0000 0011 0100 0100
0101 0110 0111 0000 0000 0000 0000 0100 0101
0110 0111 1000 0000 0000 0011 0000 0101 0101
0101 0101 0110 0111 0000 0000 0000 0011 0100

These code numbers will now become the programs which are given to the Universal Machine as input.

Machines as Programs

The code for a machine is also known as its **program**. A program is simply an algorithm written down in a language that a universal machine can interpret.



We've shown that any given Turing Machine can be turned into a program which serves as instructions for the UTM. So there is a 1-1 correspondence between machines and programs.

Up until now algorithms have been used to describe machines at a higher level of abstraction. Now they are used as something which is inputted *into* the machine. The universal machine, in effect, *reads* higher levels of abstraction.